

The Second Brain

A Personal Retrieval System with Emotion-Aware Search

Overview

This project builds a personal retrieval system that lets someone ask questions about their own life and get answers grounded in their own past writing. The system takes a person's existing personal text, such as their exported ChatGPT history and journal entries, stores it in a searchable form, and uses a language model to answer questions using the most relevant pieces of that text. This kind of system is known as retrieval augmented generation, or RAG. It is a standard and well established approach where relevant pieces of stored text are retrieved and given to a language model so its answers stay grounded in real source material rather than generic knowledge.

The main technical focus of the project is the search step. Most retrieval systems use a general purpose embedding model to decide which stored text is relevant to a question. These models are trained on broad internet data and are not especially good at recognizing emotional meaning. This project trains an embedding model to better understand emotional similarity, and tests whether it retrieves more relevant personal memories than a standard model does.

Goal

The central question of the project is simple to state. Does an embedding model that has been tuned to understand emotion retrieve more relevant personal memories than a standard off-the-shelf embedding model?

The end product is a working personal retrieval system. When the user asks something like when have I felt most uncertain about my future, the system pulls up the entries from their own history that actually carry that feeling, even when those entries do not use the same words as the question. The custom embedding model is what makes that possible, and the project measures whether it genuinely works better than the standard approach.

Why This Is Worth Doing

People now have years of personal text sitting in places like their ChatGPT history without any good way to search it by meaning or emotion. Existing note tools store and retrieve text but match mostly on keywords. They cannot reliably find the entry where you were anxious about a decision unless you happened to use the word anxious.

A retrieval system that understands emotional meaning would make a person's own history genuinely useful to them. The technical contribution here is showing, with real measurements, whether emotion-tuned retrieval is an improvement over the standard method.

What The System Does

Data Input

The RAG pipeline itself is a known architecture. The contribution of this project is in one specific part of it, the embedding model that decides which stored memories are relevant to a question. The system reads a person's exported ChatGPT history as its source of text. A person's chat history contains a mix of personal conversations and task-focused ones such as homework help. The system runs each conversation through a classifier that separates the personal, emotionally meaningful content from the task content. Only the personal content is kept for the searchable memory.

Journal entries and other personal writing are added the same way. The ChatGPT export is the largest body of personal text most people already have.

Emotional Tagging

As each piece of personal text is stored, it is tagged with basic emotional information. This uses a simple two-part model of emotion, one part for whether the feeling is positive or negative, and one part for how intense it is, along with a label for the specific emotion. This tagging supports the search step and gives a way to check whether retrieval is respecting emotional meaning.

The Custom Embedding Model

This is the core of the project. A small open source embedding model is fine-tuned on publicly available emotion datasets so that it learns to place emotionally similar text close together and emotionally different text far apart. This training does not require any of the user's personal data, which means it can be built and tested early using public data alone.

Once trained, this model is used to convert the stored personal text into searchable form. When a question comes in, the same model converts the question and finds the closest stored entries.

Retrieval and Response

When the user asks a question, the system finds the most relevant stored entries using the embedding model and passes them to a language model, which writes an answer grounded in those entries. The interface shows which entries were retrieved alongside the answer, so it is always clear what the response is based on.

Evaluation

The project includes a proper evaluation. The custom embedding model is compared against a standard embedding model on two fronts. First, on public emotion data, measuring whether it correctly recognizes emotional similarity better than the baseline. Second, on the user's own retrieval, comparing whether it pulls more relevant memories for real personal questions. This produces concrete numbers rather than just a working demo.

Decision Prediction Prototype

One of the more ambitious ideas behind this project is to see whether a system built from a person's own history can predict the decisions that person would make. The full version of this is a long term study that needs months of logged decisions to be meaningful, so it is not a core deliverable for the six

week course. However, a small early prototype of it is achievable now, and it is worth including as a proof of concept.

The prototype works using decisions that are already documented in the existing ChatGPT export. Most people have talked through real decisions in their chat history, things like career choices or other personal crossroads. For the prototype, a handful of these past decisions are selected. The system is given the context leading up to a decision but not the actual choice that was made. It is then asked to predict what the person decided and why. That prediction is compared against what actually happened.

This is intentionally a small and qualitative test, not a statistical one. With only a handful of decisions it cannot make strong claims. What it can do is show where the system predicts correctly, where it gets things wrong, and what the wrong predictions reveal about the limits of working from text alone. That analysis is genuinely interesting and sets up the larger study as future work.

Importantly, this prototype does not require building anything new. It uses the same retrieval pipeline that the rest of the project already builds, simply asked to predict rather than recall. Because of that, it is a low cost addition that only gets attempted if the main retrieval system is working well by week five. It is included here as a stretch goal rather than a required deliverable.

The honest framing is that this first version will be inferior. It will be working from limited data and a short window of history. But it is a real starting point, and as more personal data is collected over time and the later components such as the temporal layer and causal graph are added, the prediction quality is expected to improve steadily. The prototype is the seed of that larger direction, not the finished version of it.

Daily Logging and Streaks

For the system to grow more useful over time it needs a steady flow of new personal entries. The challenge is that people start journaling habits and abandon them quickly. To address this, the system includes a lightweight daily logging feature designed to keep the habit going with as little friction as possible.

The approach is modeled on the Whoop journal. Alongside any free written entry, the user is shown a short list of simple yes or no questions about their day, things like whether they slept well, exercised, felt anxious, or spent meaningful time with someone. Each one is answered with a quick tick or cross. The whole check-in takes about thirty seconds and requires no writing, which makes it easy to keep up even on busy or low energy days.

A streak counter tracks consecutive days of logging. The intent of the streak is motivation, not pressure. It is designed to be forgiving, so that missing a single day does not reset everything to zero and make the user feel like they have failed. The goal is to gently reward consistency rather than punish the occasional gap.

Over time this binary check-in data builds into a useful record on its own. It can later be lined up against the written entries to surface simple patterns, for example whether mood tends to be lower in stretches where sleep was poor. For the six week scope, the feature is implemented as a basic daily check-in with a forgiving streak counter. The deeper pattern analysis on top of it belongs to future work, since it needs months of data to mean anything.

Technical Stack

Embedding model	An open source sentence-transformer model, fine-tuned on public emotion datasets
Training data	Publicly available emotion datasets such as GoEmotions
Vector storage	A vector database for storing and searching the embedded text
Language model	Claude, for generating answers from retrieved text
Backend	Python, with a simple API layer
Interface	A basic chat interface that shows answers and the retrieved entries, plus a daily check-in
Version control	Git and GitHub throughout

Six Week Plan

Week 1	Finalize proposal and design. Set up the project, version control, and the basic data ingestion for the ChatGPT export.
Week 2	Build the classifier that separates personal from task content. Get basic retrieval working with a standard embedding model as the baseline.
Week 3	Fine-tune the custom emotion embedding model on public data. Begin comparing it against the baseline.
Week 4	Add emotional tagging at ingestion. Build the daily check-in and streak feature. Build the evaluation framework. Begin documentation.
Week 5	Run the full evaluation, custom model versus baseline. If retrieval is working well, attempt the decision prediction prototype. Refine based on results.
Week 6	Final testing, finish documentation, write the final report, and prepare the prototype demonstration.

Deliverables

- A working prototype that ingests personal text and answers questions using emotion-aware retrieval
- The fine-tuned embedding model and the code used to train it
- An evaluation comparing the custom model against a standard baseline, with measured results
- A daily check-in and streak feature for ongoing data collection
- A decision prediction prototype, attempted as a stretch goal if time allows
- Source code in a documented GitHub repository with a clear README
- A final report covering what was built, what was learned, the challenges, and the results

Privacy

The system handles personal data, so privacy is built in from the start. All personal text is stored locally or in a private store controlled by the user. Only the small amount of retrieved text needed to answer a given question is sent to the language model. The system can also be run fully locally using open source models for anyone who does not want any data leaving their machine.

Future Work

Several larger features were considered and intentionally left out of the six week scope because they depend on having months of accumulated daily data to be meaningful. These remain the natural next steps after this course.

- A temporal layer that weighs recent memories more heavily and boosts older patterns that keep recurring
- A causal graph that maps how feelings, circumstances, and decisions connect over time
- A full decision prediction study, expanding the prototype into a proper evaluation once enough personal data exists
- A proactive layer that surfaces patterns and insights without being asked

These depend on long term personal data collection, which starts now but will not produce enough volume to evaluate within six weeks. Building the retrieval core first is the right foundation for all of them.

Summary

This project builds a personal retrieval system whose main contribution is an emotion-tuned embedding model, evaluated against a standard model to see whether it retrieves more relevant personal memories. It is fully buildable in six weeks, it does not depend on accumulating new personal data because it can be proven on public data and demonstrated on an existing ChatGPT export, and it lays the foundation for the larger vision that continues after the course. A small decision prediction prototype and a daily logging feature are included as early, intentionally basic versions of directions that grow stronger as more data is collected over time.